

Project Report: Bitcoin Mining

Jin Hao Dylan Loh (z5432665), Hasitha Peter Inapuri (z5518219), Qiyue Wang (z5484179) and Pyae Sone Oo (z5271704)

Bitcoin mining is a computationally intensive process in which candidate block headers are repeatedly hashed until a resulting hash satisfies a specified target. Bitcoin's proof-of-work operates on an 80-byte block header and requires the resulting hash to be below a 256-bit target, making mining dependent on performing a large number of repeated hash computations. This project investigated the acceleration of Bitcoin's double SHA-256 (SHA-256d) hashing process using the programmable logic of an AMD Kria KV260, which integrates a Zynq UltraScale+ MPSoC containing both a processing system and programmable logic.

A software miner running on the KV260 ARM processing system was developed as a baseline, while the computationally intensive nonce search was implemented as a hardware kernel on the FPGA. The design exploited the repetitive structure of SHA-256, which operates on 512-bit blocks and produces a 256-bit digest. Key optimisations included host-side midstate preprocessing, an $II=1$ pipelined hashing architecture with a rolling 16-word message schedule, 2048-nonce processing chunks, an 8-way result scan, and hybrid DSP/LUT resource mapping. The final implementation operated at 200 MHz and achieved 131.076 MH/s, compared with 0.847 MH/s for the four-thread ARM software implementation, providing a $154.71\times$ speedup. These results demonstrate that FPGA-based pipelining and hardware/software partitioning can substantially accelerate the highly repetitive SHA-256d workload.

1. Introduction

1.1 Background

Bitcoin uses a Proof-of-Work (PoW) mechanism in which miners perform repeated cryptographic hash calculations to identify a block header that satisfies the difficulty requirement of the network. For each candidate, an 80-byte Bitcoin block header is hashed twice using the SHA-256 cryptographic hash algorithm, a process commonly referred to as double SHA-256 or SHA-256d [1]. The resulting 256-bit hash is interpreted as a numerical value and must be less than or equal to the target value derived from the block's difficulty requirement [1].

A field within the block header known as the **nonce** can be modified by the miner to produce different hash outputs. If the hash generated for a particular candidate does not satisfy the target, another candidate is tested and the hashing process is repeated [1]. Since SHA-256 is designed such that changes to the input produce unpredictable changes to the resulting digest [2], there is no direct method of determining a successful nonce without testing candidates. Consequently, Bitcoin mining involves performing a very large number of SHA-256d computations.

This repetitive workload provides an opportunity for hardware acceleration. Each candidate undergoes essentially the same sequence of operations while the nonce changes between attempts. An FPGA can exploit this regular computational structure by implementing the hashing operations as dedicated hardware and using pipelining to increase the number of

candidate nonces processed over time. This project therefore investigates the use of FPGA programmable logic to accelerate the SHA-256d nonce-search workload.

1.2 Project Aim

The aim of this project was to implement and evaluate a hardware-accelerated Bitcoin miner using the programmable logic of the AMD Kria KV260. The KV260 contains a Kria K26 System-on-Module based on the AMD Zynq UltraScale+ MPSoC architecture, which combines an embedded processing system (PS) with tightly integrated programmable logic (PL) [3]. This architecture allows the mining workload to be partitioned between software running on the ARM processor and a dedicated hardware accelerator implemented in the FPGA.

A software-only Bitcoin miner was first developed to provide a functional and performance baseline against which the hardware implementation could be evaluated. A SHA-256d mining kernel was then developed using Vitis HLS and deployed to the programmable logic of the KV260. The computationally intensive nonce search was assigned to the FPGA, while the ARM processing system was responsible for preparing the input data, initiating the accelerator and verifying the returned results.

The major objectives of the project were to:

- develop and validate a functional software implementation of the Bitcoin mining algorithm;
- implement the SHA-256d nonce-search workload as a hardware accelerator using Vitis HLS;
- partition the system such that setup and verification are performed by the ARM processing system while repeated hashing is performed within the programmable logic;
- optimise the hardware implementation to maximise hashing throughput;
- compare the performance of the FPGA implementation against the software baseline; and
- evaluate the FPGA implementation in terms of performance, resource utilisation and power consumption.

The resulting architecture therefore follows a HW/SW co-design approach, with the ARM processing system handling block-header construction, target generation, preprocessing, accelerator control and result verification, while the FPGA performs the repeated SHA-256d computations and target comparisons.

1.3 Performance Target

An initial performance target was established to provide a measurable objective for the hardware acceleration. The project aimed to achieve an FPGA hash rate of at least 20 times that of the software implementation. This was treated as a working target that would be evaluated and refined after profiling the software and hardware implementations.

Performance was primarily evaluated using hashing throughput, measured in hashes per second, with the hardware speedup determined relative to the software baseline. Resource utilisation and power consumption were also considered to determine the hardware cost associated with increasing throughput, due to the KV260's limited hardware resources. The

final implementation substantially exceeded the original performance target; however, the measured results and the resource-performance trade-offs that enabled this result are presented and analysed later in this report.

1.4 Report Structure

The remainder of this report is organised as follows. **Section 2** provides the technical background of the Bitcoin mining algorithm and SHA-256d operation. **Section 3** describes the software baseline implementation and its validation. **Section 4** presents the HW/SW architecture and the partitioning of tasks between the ARM processing system and FPGA programmable logic. **Section 5** discusses the HLS optimisation process and the techniques used to improve hashing throughput. **Section 6** describes the final hardware implementation and its verification on the KV260. **Section 7** presents the synthesis, performance, resource utilisation and power results. **Section 8** discusses these results and the associated performance-resource trade-offs. **Section 9** outlines the major challenges and limitations encountered during development, while **Section 10** identifies potential future improvements. Finally, **Section 11** summarises the outcomes of the project and presents the overall conclusions.

2. Background and Algorithm

2.1 Bitcoin Mining

Bitcoin mining is the process used to search for a block header whose hash satisfies the Proof-of-Work target specified by the Bitcoin network. The input to the mining process is an **80-byte block header**, which contains six fields: the version, previous block hash, Merkle root, timestamp, difficulty target (**nBits**), and nonce [1]. Of these fields, the 32-bit nonce can be varied by the miner to generate different candidate hashes.

For each candidate nonce, the miner constructs the block header and applies the SHA-256 hash function twice. The resulting 256-bit value is then compared against the target derived from the **nBits** field. A candidate is valid when its resulting hash, interpreted according to Bitcoin's Proof-of-Work comparison, satisfies the required target [3]. If the candidate does not satisfy the target, another nonce is tested and the process is repeated.

The mining process used in this project is illustrated in **Figure 1**, which shows the SHA-256d mining flow used as the basis of the software and hardware implementations in this project.

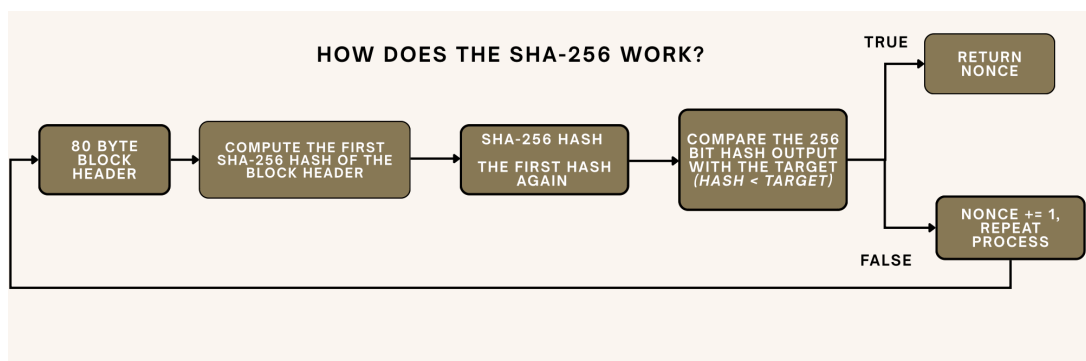


Figure 1: General SHA-256d nonce-search process used by the Bitcoin miner.

2.2 SHA-256

SHA-256 is a cryptographic hash algorithm defined by the NIST Secure Hash Standard (FIPS 180-4). It accepts an input message and produces a fixed-length 256-bit message digest [2]. Before hashing, the input is padded and divided into 512-bit message blocks, with each block processed by the SHA-256 compression function.

Each 512-bit block is initially represented as sixteen 32-bit words. These words form the beginning of the SHA-256 message schedule. The complete schedule consists of 64 words, with words 16 through 63 derived from previously generated words using logical right shifts, rotations and modulo-(2³²) addition [2]. In the standard algorithm, the expansion is defined as:

$$W_t = \sigma_1(W_{(t-2)}) + W_{(t-7)} + \sigma_0(W_{(t-15)}) + W_{(t-16)}$$

for $16 \leq t \leq 63$, where σ_0 and σ_1 consist of specified rotation and shift operations [2].

SHA-256 maintains an internal state consisting of eight 32-bit working variables, conventionally denoted (a,b,c,d,e,f,g,h). At the beginning of the hashing operation, the hash state is initialised using eight predefined 32-bit constants specified by the SHA-256 standard [2].

Each 512-bit block then undergoes 64 compression rounds. During each round, intermediate values are calculated using the current working variables, one word from the message schedule and one of 64 predefined round constants. Two important logical functions used during these rounds are the Choose and Majority functions:

$$Ch(x, y, z) = (x \wedge y) \oplus (\neg x \wedge z)$$

$$Maj(x, y, z) = (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z)$$

The compression function also uses rotations and XOR operations through the SHA-256 Σ_0 and Σ_1 functions. Together, these operations are used to calculate two temporary values during each round:

$$1 - T_1 = h + \Sigma_1(e) + Ch(e, f, g) + K_t + W_t$$

$$2 - T_2 = \Sigma_0(a) + Maj(a, b, c)$$

The eight working variables are then updated using (T₁) and (T₂). This process is repeated for all 64 rounds. After the final round, the working variables are added to the previous hash state to form the updated intermediate hash value. Once all message blocks have been processed, the eight 32-bit hash words are concatenated to produce the final 256-bit SHA-256 digest [2].

The repeated and highly regular sequence of arithmetic and logical operations within these 64 rounds is particularly relevant to the hardware implementation developed in this project. The same basic operations must be performed for every candidate nonce, allowing the SHA-256 compression process to be mapped to a pipelined hardware architecture.

2.3 Double SHA-256

Bitcoin does not use a single SHA-256 operation for block-header Proof-of-Work. Instead, SHA-256 is applied twice to the 80-byte block header [1]. This operation is commonly referred to as double SHA-256 or SHA-256d:

$$H = SHA256(SHA256(BlockHeader))$$

The first SHA-256 operation processes the 80-byte block header and produces a 256-bit intermediate digest. This digest then becomes the input to a second SHA-256 operation, producing the final 256-bit hash used for the Proof-of-Work comparison.

Consequently, every candidate nonce tested by the miner requires the SHA-256d operation to be evaluated before it can be determined whether the candidate satisfies the target. In the implementation developed for this project, each candidate nonce therefore passes through the required SHA-256 compression operations before its final digest is compared against the supplied 256-bit target. The FPGA implementation performs these candidate evaluations in chunks of up to 2048 nonces.

2.4 Computational Bottleneck

The computational cost of the mining process is dominated by repeated SHA-256d calculations. When searching across candidate nonces, most of the 80-byte block header remains constant while the nonce changes between attempts. Despite this similarity between candidates, each candidate must ultimately produce a new final hash that can be tested against the target.

This creates a highly repetitive computational workload. A potentially large nonce search requires the same SHA-256 round operations to be executed repeatedly, while comparatively little work is required for tasks such as constructing the input, controlling the search and reporting a successful result. Profiling of the project implementation therefore identified SHA-256d hashing as the primary computational workload, motivating the decision to assign repeated hashing to the FPGA while retaining setup and result verification on the ARM processing system.

The structure of the Bitcoin block header also creates an opportunity to eliminate some redundant computation. In this project, the first 64 bytes of the header are independent of the nonce. The SHA-256 state produced after processing this first 512-bit block can therefore be calculated once on the host and reused for subsequent candidate nonces. This intermediate state, referred to as the midstate, is supplied to the FPGA together with the remaining fixed header data and target.

The FPGA can then concentrate on processing the changing portion of the input and evaluating candidate nonces. Combined with hardware pipelining, this allows repeated SHA-256 operations to overlap rather than requiring one complete candidate calculation to finish before beginning the next. The repetitive and regular nature of SHA-256d therefore makes the nonce-search workload well suited to FPGA acceleration and forms the basis of the HW/SW architecture described in the following sections.

3. Baseline Implementation and Evaluation

3.1 Purpose

A software-only Bitcoin miner was developed as the baseline implementation before introducing FPGA acceleration. The baseline served two main purposes: establishing the correct functional behaviour of the mining algorithm and providing a performance reference against which the hardware-accelerated implementation could be evaluated.

Correctness was needed as subsequent HLS optimisations modified the execution structure of the SHA-256d computation. The software implementation therefore provided a reference result that could be compared with the output produced by the FPGA. A candidate nonce returned by the hardware implementation could also be independently verified in software by reconstructing the corresponding block header, calculating its SHA-256d hash and checking the result against the target.

The software baseline additionally provided a means of identifying the computational bottleneck before partitioning the application between the processing system and programmable logic.

3.2 Software Architecture

The baseline miner was implemented entirely in software, with the SHA-256d computation and nonce search executed on the processor. It accepts an 80-byte Bitcoin block header, a target value and a specified nonce search range. The implementation can be divided into three main components: block-header preparation, SHA-256d hashing and nonce-search control.

The hashing component implements the SHA-256 compression operations described in **Section 2.2**. This functionality is used to perform SHA-256d on each candidate block header. The mining component controls the search by iterating through the requested nonce range, updating the nonce field for each candidate and invoking the SHA-256d implementation. The resulting digest is compared against the supplied target, with the search terminating when a valid candidate is identified or when the requested nonce range has been exhausted.

This separation between the mining control logic and the computationally intensive hashing operation was also useful when developing the accelerated implementation. Profiling identified SHA-256d as the dominant workload, while comparatively little computation was associated with preparing the input and handling the result. Consequently, the later HW/SW implementation retained setup and result verification in software while moving the repeated SHA-256d nonce search into the FPGA programmable logic.

3.3 Baseline Validation

Before benchmarking the software baseline, its output was validated to ensure that it correctly implemented the required hashing and nonce-search operations. The SHA-256 implementation was first tested using inputs with known expected hashes. The hashes

produced by the software were compared with these expected values to confirm that the SHA-256 implementation was functioning correctly.

The complete SHA-256d operation was then tested using known Bitcoin block-header data. The block header was hashed twice and the resulting hash was compared with the expected reference value.

Finally, the complete mining operation was tested using a predefined target and nonce search range for which the expected successful nonce was known. The nonce returned by the software miner was compared with this expected result. Together, these tests verified the SHA-256 implementation, double-hashing operation and nonce-search functionality before the software implementation was used as the reference for evaluating the FPGA accelerator.

Validation Test	What is Being Verified	Expected Result	Actual Result	Status
SHA-256 test vector ("abc")	Correctness of SHA-256 implementation	Known SHA-256 digest	Calculated digest matches expected digest	Pass
Bitcoin genesis block header	Correctness of SHA-256d on an 80-byte Bitcoin header	Reference genesis block hash	Calculated hash matches reference	Pass
Nonce search	Correctness of complete mining/search process	Nonce = 8603	Nonce = 8603	Pass

3.4 Baseline Performance

After validating the software implementation, the initial FPGA kernel was benchmarked against the four-thread ARM software implementation to establish the performance prior to HLS optimisation. Both implementations processed the same workload of **2,000,000 nonces**. The baseline FPGA kernel operated at **200 MHz** and achieved a hash rate of **0.787106 MH/s**, while the ARM software implementation achieved **0.795619 MH/s**. The resulting hardware-to-software speedup was **0.989301×**, indicating that the initial FPGA implementation did not yet provide a performance advantage over the software implementation.

Metric	Baseline FPGA (PL)	ARM Software (PS)
Nonces per run	2,000,000	2,000,000

Clock frequency	200 MHz	—
Median kernel time	2.54095 s	—
Median total time (DMA + kernel)	2.54097 s	—
Hash rate	0.787106 MH/s	0.795619 MH/s
Relative speedup	0.989301×	1×

The results in the table above established the starting point for subsequent HLS optimisation, where the primary objective was to increase the throughput of the hardware hashing pipeline.

4. Hardware/Software System Design

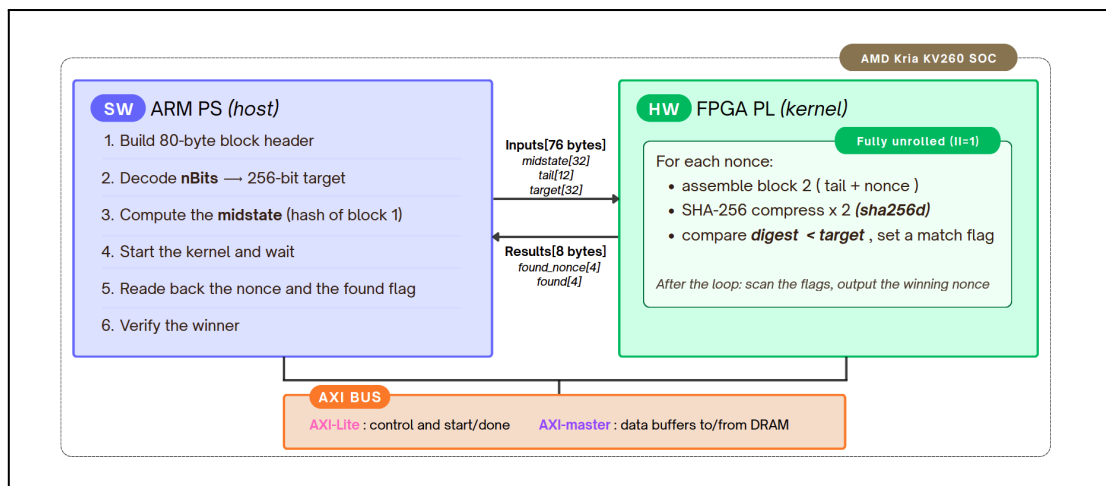


Figure 2. HW/SW Architecture of the miner

The KV260 pairs an ARM processing system (PS) with FPGA programmable logic (PL). We split the miner so that each side does what it is best at. The PS runs the setup and the final check, which stays constant across the searches and is easy to get wrong so it stays in the software side where it is simple to write and test. The programmable logic runs the search itself which is the same SHA256d hash over billions of candidate nonces where dedicated hardware can pipeline the work. Figure 2 shows the full system. The decision that ties together is a deliberately small interface where the PS hands the PL a tiny, fixed job and then stays out of the way, so the hardware is never left waiting for the data.

In the processing system (left of the figure), the host builds the 80-byte block header, decodes the compact n bits field into the 256-bit target, and computes the SHA256 midstate (next section explains why it is precomputed). Bitcoin stores its header fields in little-endian

order and reverses the displayed hashes, which is a common source of bugs, so we keep all of that byte handling in software. After the kernel returns, the host re-hashes the winning nonce on the PS to confirm the hardware found a genuine result. On the programmable logic side (right of the figure), the kernel does the repetitive part where for each nonce it assembles the changing block from the fixed tail and the nonce, runs the two SHA256 compressions on top of the midstate, compares the digest to the target and records a match. Sections 5 and 6 will cover how that loop is pipelined.

The two sides communicate over AXI (bottom of the figure). Control is small and infrequent, so the start signal and the nonce range travel over AXI-lite registers. The data buffers use AXI-master, where the kernel reads them directly from DRAM. The amount of data is small on purpose because the input is 76-bytes and return data is only about 8 bytes. Just as important, the host sends this once per sweep of many nonces, not once per nonce. Calling the kernel per nonce would let the start-up and transfer overhead dominate and erase the speedup, so a single call covers a large nonce range and the kernel reports only the result.

5. Hardware Optimisation

Optimisation Journey

Step	Changes	Effects on II / Resources	Outcome
Direct HLS Port	Software SHA256 loop moved to HLS as written	II ~231	Baseline
Midstate reuse	Host precomputes the block-1 hash and the kernel receives the midstate	Compressions reduced from 3 to 2	Kept
Pipeline the nonce loop	Target II = 1 on the hashing loop	Throughput = Clock / II	Goal Set
Full unroll (brute force)	Unroll all rounds and store the whole expanded message schedule	~ 290% LUT usage	Did not fit on KV260
Rolling 16-word schedule	Recompute schedule words in a 16-word window instead of storing all 64	II = 1 at ~ 86% LUT	Kept
Chunk & scan	Process nonce in chunks of 2048. Write a match flag per nonce. Scan 8 flags per cycle after the loop	Keeps II = 1	Kept
Pure LUT mapping	Remove all DSP bindings	~ 97% LUT, no routing headroom	Failed Timing
DSP-heavy mapping	Map the adders onto DSP blocks	Fits but fails 200 MHz	Rejected
Hybrid DSP/LUT & physopt	Balance adders across DSP and fabric. Post-route physical optimisation	Meets 200 MHz	Final Design

5.1 Why Initiation interval, not latency

The natural instinct is to make each hash finish sooner, that is to cut its latency. For a pipeline, this is the wrong target. Latency is a fixed startup cost paid once where throughput depends on how often a new nonce can enter, which is the initiation interval (II). One SHA256d still takes many cycles to travel through the pipeline but with II value of 1, a new nonce starts every cycle, like an assembly line so the work into lowering the II value and not shortening a single hash.

5.2 Midstate Reuse

The nonce sits in the last four bytes of the 80-byte header (76 down to 79) and SHA256 processes data in the 64-byte blocks. The first block, bytes from 0 to 63, therefore never change while the nonce is swept. We hash that block once on the host and pass the result, the midstate, to the kernel. Each nonce then needs only the second block of the first hash and the full second hash where two SHA256 compressions instead of three. This removes a third of per-nonce work before any hardware tuning.

5.3 Reaching $II=1$ within the device

Unrolling every round to reach $II=1$ is simple in principle but expensive. The naive version lays out the full 64-word expanded schedule at every stage, which pushed LUT use to roughly 290% of the device so it could not be built. The fix was a rolling schedule where instead of storing all 64 words, the kernel kept a 16-word window and computed each new schedule word on the fly. This maintains $II=1$ and drops LUT use to about 86%.

There are two other steps which make it possible to maintain $II=1$. First, the kernel processes nonces in chunks of 2048. Each nonce writes a one-bit match flag, and a separate pass scans the flags eight at a time after the chunk. Writing a flag rather than stopping on a hit means the pipeline never waits on the output, which is what protects the $II=1$ rate. Second, the design would not close 200 MHz timing, with every adder in fabric (about 97% LUT usage, and no routing headroom), and moving every adder onto DSP blocks created long DSP critical paths that also failed. The final design balances the adders across DSP and fabric and uses post-route physical optimisation to meet the 200 MHz clock.

The result is a single hashing engine at $II=1$ and 200 MHz, using about 86% of the LUTs, 40% of the DSPs, 22% of the flip-flops and 10% of the BRAM. Its measured throughput on the board is reported in Section 7.

6. Final Implementation

6.1 Final Mining Algorithm

The complete pseudocode for the final miner is listed below. In short, the host precomputes the midstate, the fixed tail words and the target, and the kernel then sweeps the nonce range, hashes each candidate with SHA256d, and returns the first nonce whose digest falls below the target.

6.2 Final HLS architecture

The final kernel is a single SHA256d engine clocked at 200 MHz. The 64 hashing rounds are unrolled into a straight pipeline, and the message schedule uses a rolling 16-word window instead of the full expansion, which is what lets the pipeline run at $II=1$ while still fitting the device. Nonces are processed in chunks of 2048 where each nonce writes a one-bit match flag and an 8-way scan reads those flags after the chunk. The adders are split across DSP blocks and fabric, and post-route physical optimisation closes the 200 MHz timing.

6.3 Correctness Verification

Since the kernel is a tuned pipeline, we checked its output against a trusted reference at three levels.

First, the software baseline from Section 3 acts as a golden model where for a set of known headers, the FPGA digests must match the software digests bit by bit. Secondly, we run the search with an easy target whose top two bytes are zero, and the kernel must return the known winning nonce, 8603, for the genesis header. Thirdly, the host does not trust the hardware blindly. When the kernel reports a winner, the PS re-hashes that nonce in software and confirms the digest is below the target before accepting it. This same re-hash runs on the board during every measurement, so a silent hardware error would show up as a failed check rather than a wrong result reported as correct.

7. Results

The final implementation of our Bitcoin mining accelerator was evaluated using both the Vitis HLS synthesis results and measurements obtained from the output on the Kria KV260 board. The performance of our optimised design was compared against the software-only SHA-256d implementation running on the ARM Processing System (PS). The results measure the pipeline throughput, execution time, hardware resource utilisation, software-to-hardware speedup and power consumption.

7.1 HLS Synthesis Results

Metric	Clock frequency	HASH_CHUNK II	BRAM	DSP	FF	LUT
Result	200 MHz	1	30 (~10%)	500 (~40%)	52378 (~22%)	101091 (~86%)

The bitcoin miner was synthesised for a target clock frequency of 200MHz. The main HASH_CHUNK loop, responsible for processing candidate nonce values through the SHA-256d datapath, managed to achieve an optimal initiation interval (II) of 1. Having an II of 1 means that a new nonce candidate can enter the processing pipeline every clock cycle. Therefore, at a clock frequency of 200MHz, this will give an ideal theoretical initiation throughput of 200MH/s which means that the pipeline can process 200 million nonces per second.

It is important to distinguish this initiation rate from the latency of an individual SHA-256d calculation. An individual nonce requires multiple pipeline stages before its result becomes available. However, these stages occur simultaneously on different nonce candidates. Furthermore, after the pipeline has been filled, the accelerator does not need to wait for one nonce to complete before beginning the next.

The final best design achieved this performance with a relatively high resource utilisation. The synthesis reported approximately 30 BRAMs (10%), 500 DSPs (40%), 52378 FFs (22%) and 101091 LUTs (86%). Based on these utilisation results, LUT utilisation is the primary resource constraint of our final implementation. This high utilisation reflects the trade-off made between FPGA resource utilisation and performance. The design uses a deeply pipelined SHA-256d datapath so that nonce processing can be initiated every cycle. This increases the amount of resources required but it substantially increases the throughput.

7.2 KV260 Performance

Metric	Nonces/run	Median kernel time	FPGA hash rate	ARM hash rate	Speedup
Result	268435456	2.048s	131.076 MH/s	0.847 MH/s	154.71x

Hardware throughput was calculated using:

$$\text{Throughput} = \text{nonces/kernel time, which leads to } 268435456/2.048s = 131.076 * 10^6$$

Therefore, the final design managed to achieve a throughput of 131.076 MH/s on the board.

7.3 Software baseline

A software-only SHA256d implementation was also executed on the KV260's ARM processing system to provide a baseline for comparison. The software implementation used four ARM threads and processed 2000000 nonce candidates in 2.36s. The corresponding software throughput was:

$$2000000/2.360653s = 0.847 * 10^6 \text{MH/s}$$

8. Discussion

8.1 Performance

Comparing the results of the optimised implementation and the software baseline implementation, the optimised implementation achieved a hardware speedup of 154.71 times over the software baseline implementation.

$$\text{Speedup} = 131.076/0.847 = 154.71$$

8.2 Theoretical and Measured throughput

Our board displayed that our final implementation achieved a throughput of 131.076 MH/s. However, we initially calculated that the Main HLS pipeline has a theoretical throughput of 200 MH/s, based on the 200 MHz clock frequency and the SHA_CHUNK initiation interval of 1. Therefore, the throughput we achieved on the board corresponds to roughly 65.5% of the theoretical pipeline throughput.

$$131.076/200 * 100 = 65.5\%$$

This result does not imply that the $II = 1$ result is incorrect. The reason for the difference in results could be because the full design performs additional operations beyond initiating hashes. This includes processing finite chunks of nonce candidates, recording comparison results, scanning the results for successful nonces and performing loop and control operations between chunks. During these stages, the main hashing pipeline cannot necessarily sustain its ideal initiation rate continuously. This affects the throughput output, resulting in a throughput lower than the theoretical value.

8.3 Performance vs Resource Trade-off

Our final implementation required significant FPGA resources. In particular, the final implementation consumed approximately 86% of the available LUT resources. This makes

LUT usage the primary resource limitation of our best design. This result reflects an important hardware acceleration trade-off. Optimisation, increasing parallelism and pipelining improves performance but requires higher resource utilisation. While our design is able to run on the KV260 board and achieve optimal results, the limited remaining LUT capacity would further constrain additional expansion of our designs architecture.

8.4 Power Consumption

Power measurements were also measured from the board before and during execution of the mining accelerator. When the board is in its idle state, the total SOM power is 3.63W. When the board was running the mining accelerator, the SOM power increased to 4.47W which represents an increase in power of 0.84W, roughly 23% relative to the idle measurement.

State	Idle	Mining	Increase
Power	3.63 W	4.47 W	0.84 W

However, these measurements should be interpreted as total SOM power measurements, rather than the isolated dynamic power consumption of the FPGA mining kernel. The 0.84W increase provides an estimate of the additional system power observed while the accelerator was running.

9. Challenges and Limitations

9.1 Initial Sequential Design

The initial HLS implementation was adapted directly from the software design and achieved an initiation interval of approximately $II = 231$. Although resource usage was manageable, this severely limited throughput because a new nonce could only enter the pipeline every 231 cycles. Reducing the initiation interval therefore became the main optimisation priority.

9.2 FPGA Resource Constraints

An aggressive attempt to achieve $II = 1$ required approximately 290% LUT utilisation, making the design impossible to implement on the K26 FPGA. Later optimisations, including midstate reuse, a rolling 16-word message schedule and selective DSP use, reduced the resource requirement significantly. However, the final design still used approximately 86% of available LUTs, leaving limited headroom for additional parallelism.

9.3 Timing Closure

Meeting the required 200 MHz clock was challenging even after $II = 1$ was achieved in HLS. DSP-heavy implementations reduced LUT usage but introduced critical timing paths, while removing DSP bindings increased LUT utilisation to approximately 97%. The final design therefore used a hybrid DSP/LUT mapping together with post-route physical optimisation to achieve the required clock frequency. This showed that good HLS results do not necessarily guarantee successful placement and routing.

9.4 Benchmarking Overhead

Short FPGA benchmarks produced misleadingly low throughput because XRT launch and control overhead dominated execution time. Larger nonce ranges were therefore required to better represent sustained accelerator performance. The final benchmark used 268,435,456 nonces per run, achieving approximately 131.076 MH/s.

9.5 Scalability

Although the final pipeline operates at $II = 1$, further scaling is limited by high LUT utilisation and routing congestion. Multiple hashing pipelines could theoretically increase throughput, but additional resource optimisation would be required before such replication could fit and meet timing on the KV260.

10. Future Improvements

10.1 Earlier Profiling

Future development should use HLS scheduling, resource and timing reports earlier to identify the main bottlenecks before applying aggressive optimisations. This would reduce time spent exploring designs that cannot meet resource or timing requirements.

10.2 Additional SHA-256 Precomputation

More nonce-independent SHA-256 values could potentially be precomputed on the ARM processor. In particular, parts of the second-block message schedule may be reusable across nonce attempts, reducing FPGA arithmetic and resource usage.

10.3 Multiple Hashing Pipelines

If resource utilisation can be reduced, multiple independent hashing pipelines could be investigated. This would allow more than one nonce to be processed per clock cycle and potentially increase throughput beyond the current single-pipeline $II = 1$ design.

10.4 Timing-Aware HLS Optimisation

Physical timing should be considered earlier during HLS optimisation. DSP and LUT mappings could be evaluated using routed timing reports throughout development rather than only after the final HLS architecture has been selected.

10.5 Improved Benchmarking

A more detailed benchmarking framework could separately measure host preprocessing, kernel launch, FPGA execution and data-transfer overhead. Comparing against a more optimised ARM implementation would also provide a stronger software baseline.

10.6 Automated Design-Space Exploration

Parameters such as chunk size, scan width, pipeline structure and DSP mapping were explored manually. Automating this process could identify better performance/resource trade-offs and reduce development time.

11. Conclusion

This project successfully implemented and evaluated a hardware-accelerated SHA-256d Bitcoin miner on the AMD Kria KV260. The workload was partitioned between the ARM Processing System, which performs preprocessing and setup, and the FPGA Programmable Logic, which performs the computationally intensive nonce search and hashing.

Key optimisations included SHA-256 midstate reuse, a rolling 16-word message schedule, chunk-based nonce processing and an $II = 1$ hashing pipeline. Achieving this performance required careful balancing of LUT and DSP utilisation, with the final hybrid implementation successfully operating at the required 200 MHz.

The final design was verified on hardware using known test data and correctly returned the expected nonce of 8603. It achieved approximately 131.076 MH/s, compared with 0.847 MH/s for the four-thread ARM implementation, giving a speedup of approximately $154.71\times$ and exceeding the original $20\times$ performance target.

The main limitation is the high LUT utilisation of approximately 86%, which restricts further scaling. Overall, the project demonstrated that FPGA acceleration can provide substantial performance improvements for highly repetitive SHA-256d workloads, while also highlighting the importance of balancing throughput, resource utilisation and physical timing.

References

- [1] "Block Chain — Bitcoin." Accessed: Aug. 14, 2026. [Online]. Available: https://developer.bitcoin.org/reference/block_chain.html
- [2] W. May, "FIPS PUB 180-4 FEDERAL INFORMATION PROCESSING STANDARDS PUBLICATION Secure Hash Standard (SHS) CATEGORY: COMPUTER SECURITY SUBCATEGORY: CRYPTOGRAPHY," *Secure Hash Standard (SHS)*, 2015, doi: 10.6028/NIST.FIPS.180-4.
- [3] AMD, "Kria K26 SOM Data Sheet," DS987, Advanced Micro Devices, Santa Clara, CA, USA, 2022. [Online]. Available: <https://docs.amd.com/r/en-US/ds987-k26-som>. [Accessed: 13 Aug. 2026].
- [4] AMD, "Vitis High-Level Synthesis User Guide," UG1399, Advanced Micro Devices, 2025. [Online]. Available: <https://docs.amd.com/r/en-US/ug1399-vitis-hls>. [Accessed: 13 Aug. 2026].
- [5] AMD, "Vivado Design Suite User Guide: Implementation," UG904, Advanced Micro Devices, 2025. [Online]. Available: <https://docs.amd.com/r/en-US/ug904-vivado-implementation>. [Accessed: 13 Aug. 2026].
- [6] Bitcoin Wiki, "Block hashing algorithm." [Online]. Available: https://en.bitcoin.it/wiki/Block_hashing_algorithm. [Accessed: 13 Aug. 2026].
- [7] R. Kastner, J. Matai, and S. Neuendorffer, *Parallel Programming for FPGAs*, 2018. [Online]. Available: <https://pp4fpgas.readthedocs.io>. [Accessed: 13 Aug. 2026].
- [8] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>.
- [9] NIST, "Secure Hash Standard (SHS)," FIPS PUB 180-4, National Institute of Standards and Technology, Gaithersburg, MD, USA, Aug. 2015. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.180-4>.

Appendix A: Final Mining Pseudo Code

This is the final miner. The host prepares the fixed data once, then the kernel searches the nonce range on the FPGA. The SHA256 compression function it calls is listed in Appendix B.

Host (ARM PS)

```
inputs:
    header      : 80-byte block header
    target      : 256-bit target (from the decoded nBits field)
    nonce_start : first nonce to try
    nonce_count : number of nonces to try

midstate = sha256_compress(H0, header[0..63]) # first 64 bytes never change with the nor
tail_words = words(header[64..75])           # 12 fixed bytes as three 32-bit words
target_w = words(target)                     # eight 32-bit words

send (midstate, tail_words, target_w) to the kernel
start kernel with (nonce_start, nonce_count)
read back (found, found_nonce)
if found:
    re-hash found_nonce on the PS and confirm the digest is below target
```

Kernel (FPGA PL)

```
mine(midstate, tail_words, target, nonce_start, nonce_count):
    CHUNK_SIZE = 2048

    for chunk_base = 0 to nonce_count step CHUNK_SIZE:
        clear match_flags[0 .. CHUNK_SIZE-1]

        # main loop, pipelined so a new nonce enters every clock cycle (II = 1)
        for i = 0 to min(CHUNK_SIZE, nonce_count - chunk_base) - 1:
            nonce = nonce_start + chunk_base + i
            digest = sha256d(midstate, tail_words, nonce)
            match_flags[i] = (digest < target)

        # scan eight flags per cycle for the lowest winning nonce in this chunk
        for i = 0 to CHUNK_SIZE-1 step 8:
            if any of match_flags[i .. i+7] is set:
                found_nonce = nonce_start + chunk_base + (first set index in the group)
                found = 1
                return

    found = 0 # no nonce in the range met the target
    return
```

Double Hash from the midstate

```
sha256d(midstate, tail_words, nonce):
    block2 = tail_words ++ nonce ++ padding # completes the 80-byte first message
    h1 = sha256_compress(midstate, block2) # finishes the first SHA-256
    block = h1 ++ padding # 32-byte message for the second SHA-256
    return sha256_compress(H0, block) # second SHA-256, the final digest
```

In the kernel, this sha256d is not a function call but a fully unrolled hardware pipeline, so the loop above accepts one nonce per clock cycle. Sections 5 and 6 articulate that structure.

Appendix B: SHA-256 Pseudocode


```
SHA256(message):

    Pad message to a multiple of 512 bits

    Initialise hash state

    FOR each 512-bit block:

        Split block into 16 32-bit words

        FOR round = 0 TO 63:

            IF round >= 16:
                Generate next message-schedule word

            Calculate round values using:
                - rotations and shifts
                - Choose function
                - Majority function
                - round constant

            Update hash state

        Combine final hash state

    RETURN 256-bit hash
```

Appendix C: Additional HLS Results

Performance & Resource Estimates

<

>

≡

👤

⌕

🔍

🔄

%

📏

▼

☒ Modules

☒ Loops

☒ Hide empty columns

Modules & Loops	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	Structure	Bram	DSP	FF	LUT	URAM
✓ mine (1)	5.069E7		10,137,744		no	function	30	500	52,378	101,091	0
✓ CHUNK_LOOP (2)	5.069E7	2,475	-	4,096	no	loop					
✓ mine_Pipeline_HASH_CHUNK (1)	1.105E4		2,049		loop pipeline steps	function	0	500	47,695	95,148	0
> HASH_CHUNK (2)	1.105E4	163	1	2,048	yes	loop					
✓ mine_Pipeline_SCAN_CHUNK_PARALLEL (1)	1.295E3		257		loop pipeline steps	function	0	0	77	419	0
✓ SCAN_CHUNK_PARALLEL	1.285E3	3	1	256	yes	loop					

Appendix D: Validation Results

KV260 Board output

```
=====
PL HARDWARE (optimized kernel @ 200 MHz)
  nonces/run      : 268435456
  runs            : 5
  median kernel: 2.047930 s
  median total   : 2.047937 s
  hash rate      : 131.076 MH/s

ARM PS SOFTWARE (4-threaded SHA256d)
  nonces          : 2000000
  time            : 2.360653 s
  hash rate       : 0.847 MH/s
  checksum        : 0xf43022b934b0c

SPEEDUP hardware / software : 154.71x
=====
TEST PASSED
```

Board idle

```

===== 2026-08-09 05:33:45 =====
Power Utilization
SOM total power           :      3630 mW
SOM total current         :      718 mA
SOM total voltage         :     5042 mV

AMS CTRL
System PLLs voltage measurement, VCC_PSLL           :     1202 mV
PL internal voltage measurement, VCC_PSBATT          :      716 mV
Voltage measurement for six DDR I/O PLLs, VCC_PSDDR_PLL :     1793 mV
VCC_PSINTFP_DDR voltage measurement                  :      840 mV

PS Sysmon
LPD temperature measurement           :      32 C
FPD temperature measurement (REMOTE)  :      32 C
VCC PS FPD voltage measurement (supply 2)           :     845 mV
PS IO Bank 500 voltage measurement (supply 6)       :     1781 mV
VCC PS GTR voltage                   :      858 mV
VTT PS GTR voltage                   :     1799 mV

PL Sysmon
PL temperature                   :      30 C

```

Board running

```

===== 2026-08-09 05:34:43 =====
Power Utilization
SOM total power           :      4470 mW
SOM total current         :      871 mA
SOM total voltage         :     5041 mV

AMS CTRL
System PLLs voltage measurement, VCC_PSLL           :     1202 mV
PL internal voltage measurement, VCC_PSBATT          :      718 mV
Voltage measurement for six DDR I/O PLLs, VCC_PSDDR_PLL :     1791 mV
VCC_PSINTFP_DDR voltage measurement                  :      841 mV

PS Sysmon
LPD temperature measurement           :      32 C
FPD temperature measurement (REMOTE)  :      32 C
VCC PS FPD voltage measurement (supply 2)           :     840 mV
PS IO Bank 500 voltage measurement (supply 6)       :     1780 mV
VCC PS GTR voltage                   :      857 mV
VTT PS GTR voltage                   :     1796 mV

PL Sysmon
PL temperature                   :      32 C

```